

RobotStudio: Exercise 1

Using a pen tool

Applied Robotics & Applied Robotics for Architects
FRTN85/FRTN80
Faculty of Engineering, Lund University

August 2026

Contents

1	Introduction	2
1.1	Prerequisites	2
1.2	Launching RobotStudio	4
1.3	Try out some pre-made stations	5
1.4	Checking if the license works	7
1.5	Taking a look around	8
1.6	Running a simulation	8
2	The pen drawing station	9
2.1	Creating a solution file	9
2.2	Importing objects	10
2.3	Defining the home position	12
2.4	Defining a workobject	16
2.5	Creating targets and paths	19
2.6	Creating a RAPID program	21
2.7	Analyzing and editing RAPID code	21
2.8	Simulation options and an additional assignment	28
3	Testing your program on the real robot	29

1 Introduction

The aim of this exercise is to familiarize yourself with RobotStudio, the software in which we program the robots that are utilized later on in the course. In this document you will go through all of the parts of the general robot programming workflow. The exercise will consist of the following main steps:

- **creating a workstation**
- **defining the move instructions**
- **creating a simulation**
- **editing RAPID code** (code that enables communication with the targeted robot)
- **uploading the code to the target robot** (physical robot that is located in the Robot-Lab) via a USB-drive to test it out

In particular, you will program the robot (which is a robotic arm) to write with a pen on a paper.

The software used is for advanced robot programming and simulation. Please read the instructions **carefully**, there are many hints on how to operate the software. The instructions for this exercise are very detailed. The instructions for Exercise 2 and 3 will be less detailed as you get more used to the software.

Important tips will be placed in boxes like these ones. However, we strongly recommend that you **read everything** — the instructions are detailed and made to guide you step-by-step.

Save your work **FREQUENTLY!** The program is loaded through network drives and it can stop working unexpectedly.

1.1 Prerequisites

Before launching RobotStudio, create two **local** folders, namely:

- `C:\Temp\Documents\yourName\Robotstudio`
- `C:\Temp\Documents\yourName\Robotstudio\Projects`

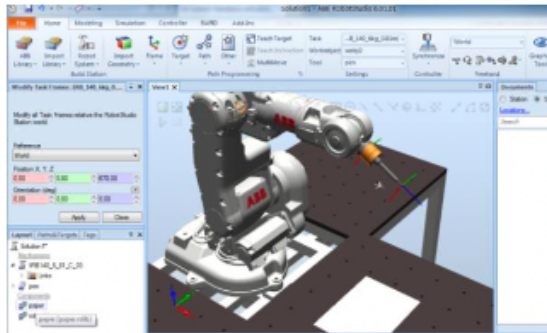
You should keep all RobotStudio files on the **local machine (computer)**. RobotStudio does not support network mounted folders (e.g., such starting with `\\shared` or `\\temp`). RobotStudio supports multiple methods of saving your work (you will later learn about the differences).

As long as you are in a session and have RobotStudio open, it is enough to use the "save" button without giving it too much thought. That being said once you finish, you should use the *Pack and Go* option which can be found under *File* and then *Share*, and then move the pack and go station to either your home folder or a USB drive.

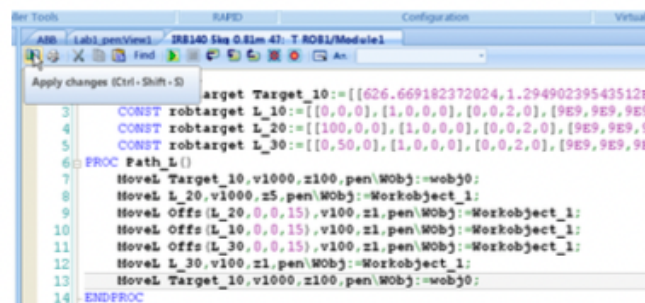
The next time you start the session, you will open the *Pack and Go* file and begin.

Always save your files to a local folder!

When starting or ending a session, use a *Pack and Go* file!

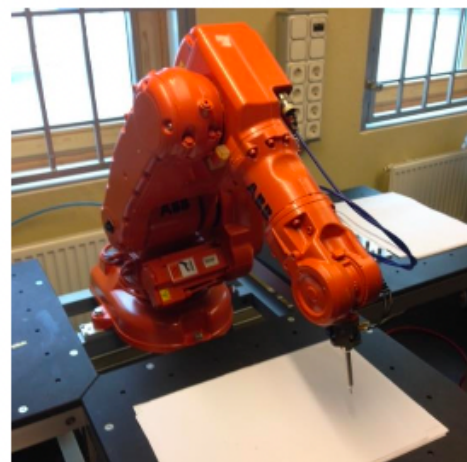


1 Modeling and simulation in program RobotStudio



2 RAPID-program auto-generated from your specified target-points and paths in RobotStudio

(or handwritten in text-editor or programmed at Flexpendant): instructions for robot motion



3 Download and test your RAPID-program on a real robot (ABB IRB140)

1.2 Launching RobotStudio

After you have created your local folders, start RobotStudio using the LU app launcher which can be found at <https://apps.lu.se>. Launch the app called **RobotStudio 2025**.

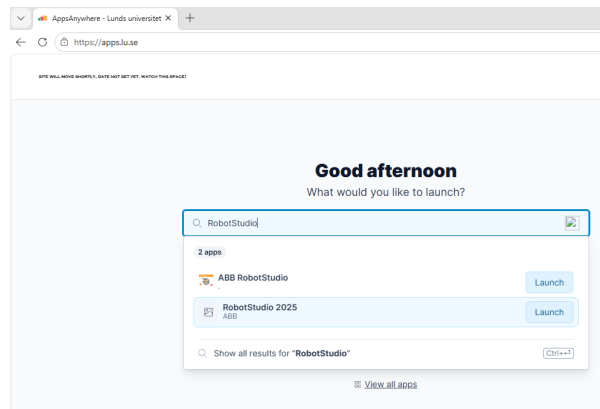


Figure 1: Starting RobotStudio from a web browser (use your STiL-account)

Make sure you do not accidentally start two instances of RobotStudio. There is **only one** license per computer, which leads to saving being disabled in the second window.

Make sure to launch the app called *RobotStudio 2025*, and **not ABB RobotStudio**.

Immediately after starting, change the default options of the program to save progress locally. **As we have already stated, RobotStudio does not work with online folders.** Click on *Options* on the side menu (location of which is indicated in Figure 2), and in

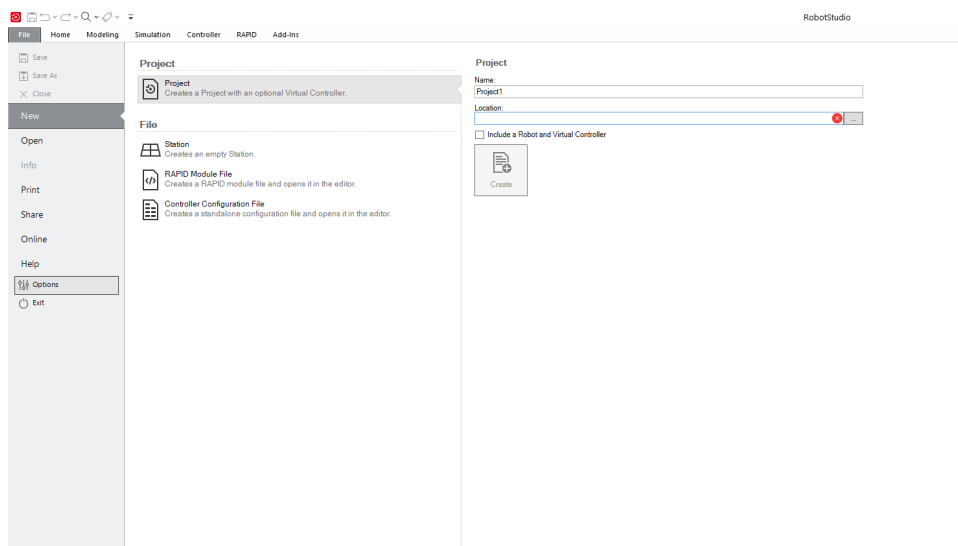


Figure 2: Selecting *Options* on the side menu

the pop-up window (shown in Figure 3) make sure the user document and solutions directories are set to the local folders you have created in the prerequisites step.

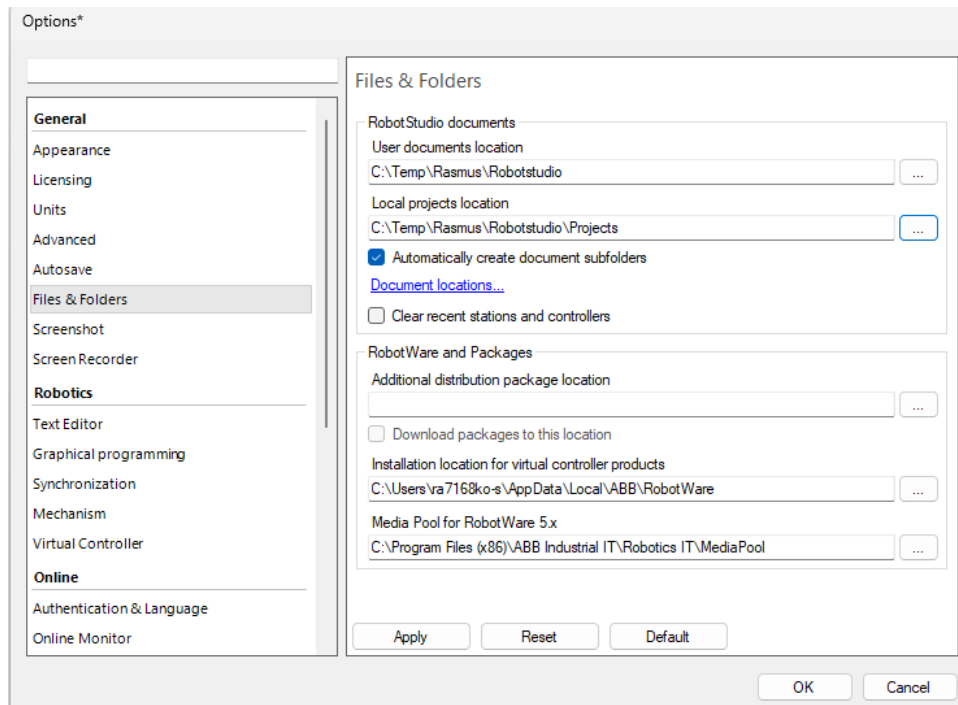


Figure 3: Configuring the working folder options

1.3 Try out some pre-made stations

Once you have made sure everything works, it is time to start exploring the pre-made examples. Go to **File → Open** and **click on Demo Stations** where all the sample stations are located. Open any of these. We will describe the **"Demo FlexLoader"** station. Click **Next** on the wizard that pops up. Type in the address (path) of a **new and empty** folder in the *Target folder* field (for an example refer to Figure 4). Remember to use your local folders, and not the network ones. (The paths might already be filled in, these probably work)

You can only unpack workstations into empty folders, RobotStudio will not let you unpack into a non-empty one.

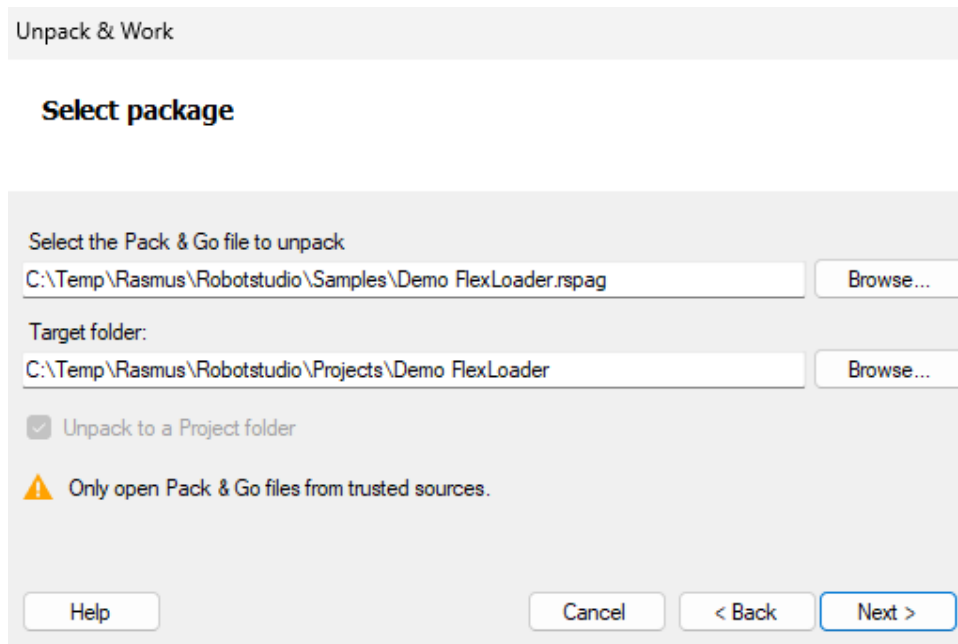


Figure 4: Unpack the workstation into an empty folder

After you have clicked *Next* you will have to choose the RobotWare version. If only RobotWare version 7.xx is installed you will get an error as in Figure 5. We need

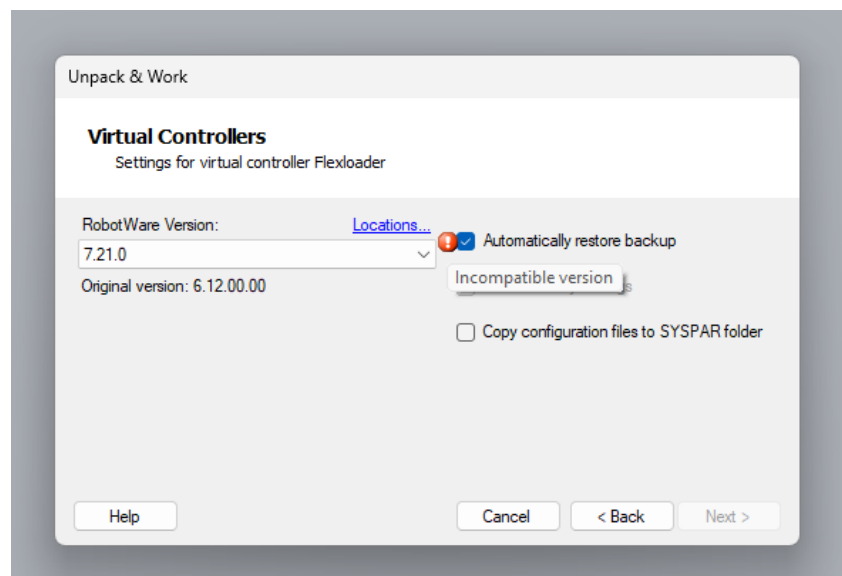


Figure 5: Incorrect RobotWare Version installed

version 6.xx which may not be installed by default. However this can be done by first **cancelling the current unpacking**, then going to the **Add-Ins** tab at the top. Click on "Gallery" to get the same view as in Figure 6. Click on **RobotWare for IRC5**, then on the right side of the screen, choose the latest 6.xx version and click on "Add". Accept the following prompts to install the required RobotWare Version. When this is done you can continue exploring the Demo Station.

Now, when we have the appropriate RobotWare Version, once again open the Demo Station as previously explained, and continue past Figure 5 with *Next* → *Finish*,

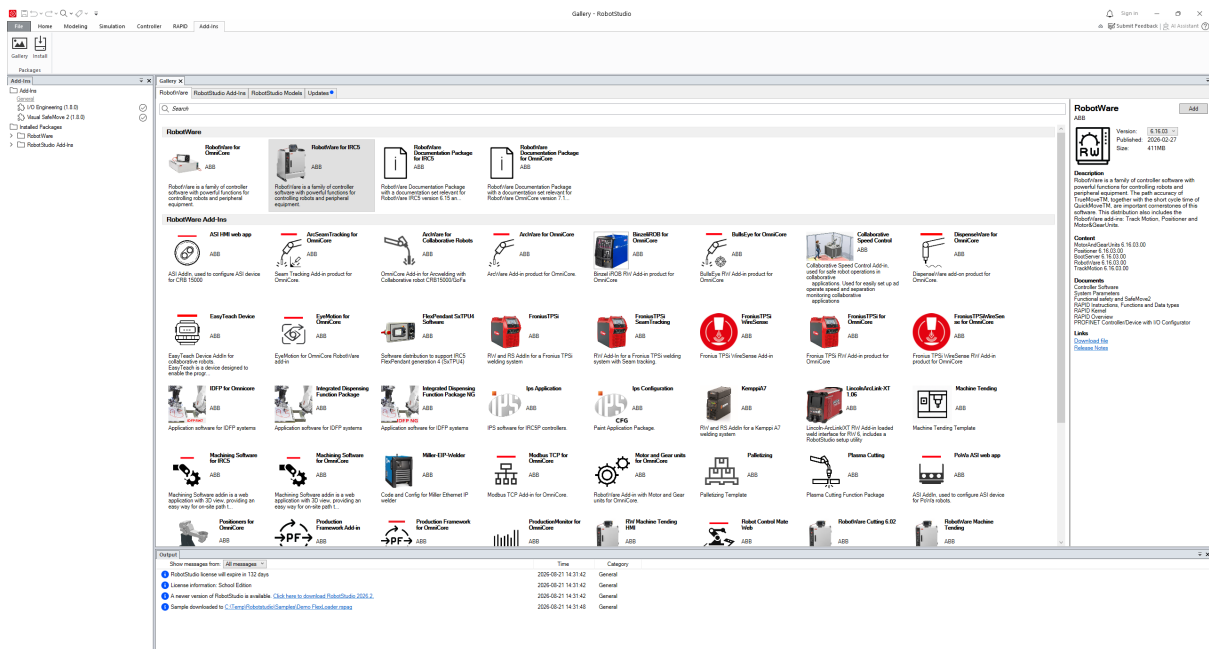


Figure 6: Install RobotWare

wait for the station to load, and then **wait** for the “Controller status” in the bottom right corner to **turn green**, as indicated in Figure 7. You have now loaded your first workstation!

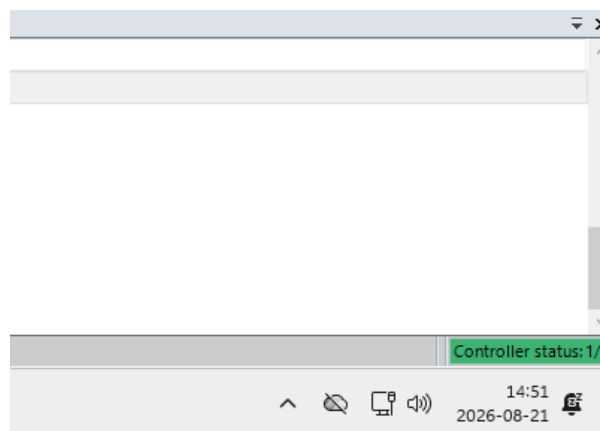


Figure 7: Wait for the controller status to turn green! (bottom right corner)

Wait for the controller status to turn green!

1.4 Checking if the license works

RobotStudio is licensed to one instance per computer. This means you get **only one** open window! If you accidentally open two and start working in the “wrong” one, you will not be able to save your progress — and you will not be able to “copy-paste” it into a working instance either.

Before doing anything, make sure the licensing is okay. The easiest way to check this is to make a small change to the Demo Station and then try to save. For example,

try deleting one of the yellow-ish objects by right clicking on it and then press delete. Look at the save icon. If it is not greyed out and you can save normally, you have a working license.

Before doing anything else, ensure that the “save” feature works! Otherwise you will lose all your recent work.

1.5 Taking a look around

Try out the following mouse functions to get used to them.

Pan by holding Ctrl + Left Mouse Button and dragging.

Zoom by scrolling with Middle Mouse Button, or press the Middle Mouse Button and horizontally drag.

Rotate by holding Ctrl + Shift + Left Mouse Button.

You can also right click anywhere in the simulation window and use the predefined views *View all* or *View center*.

1.6 Running a simulation

Once you have taken a look around, try running a simulation! Move over to the Simulation tab (shown in Figure 8) and select *Play*. You can move around during the simulation to get a better look.

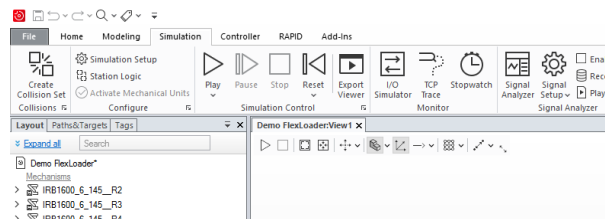


Figure 8: Using the Simulation tab

This concludes the introductory part, you are now all set to start making your first station in RobotStudio! Make sure to remember the hints outlined in boxes — check your license, save your progress and use offline folders!

2 The pen drawing station

2.1 Creating a solution file

The goal now is to recreate **the robot** station you will be using during the hands-on part of the laboratory in simulation. Your workstation will consist of an ABB 140 robot, a table and a pen tool.

Start by going over to the *File* tab and selecting *New* → *Project*, check the box "Include a Robot and Virtual Controller" as indicated in Figure 9. Change the robot model to the **IRB140 6kg Type C** robot (**choose the model without the T**), type in a location (path) and click *Create*!

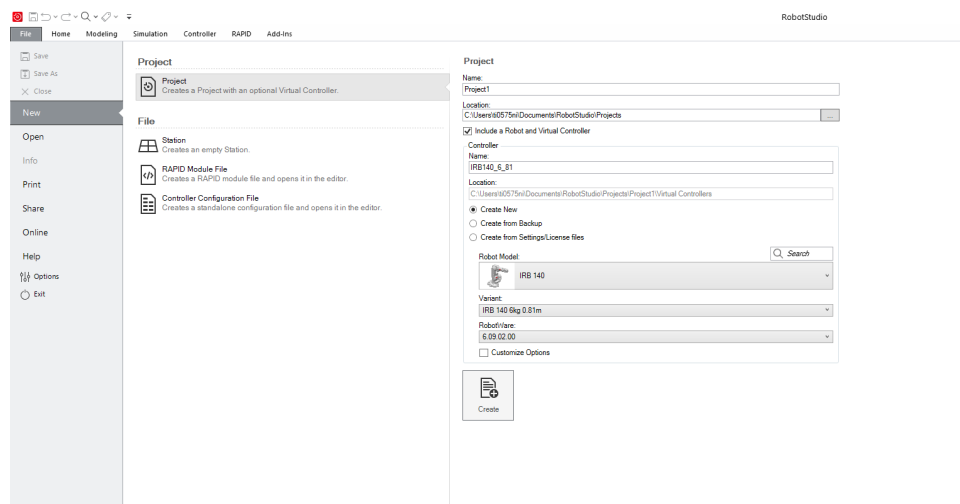


Figure 9: Creating the solution file

Once loaded, RobotStudio will once again ask you which model of robot to import. Choose the version **without the T**.

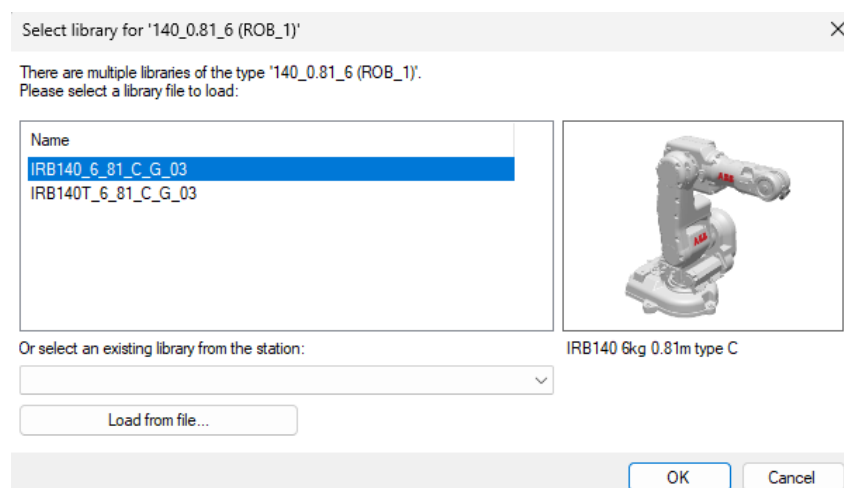


Figure 10: Select the correct robot version IRB140_6.81.C.G.03

Loading everything into the simulator will take some time. Just as before, you have to **wait** for the Controller indicator to go green (it is located at the bottom right corner).

2.2 Importing objects

Once the *Controller status* goes green, look for the *Import library* button, usually found in the upper left corner, indicated in Figure 11.

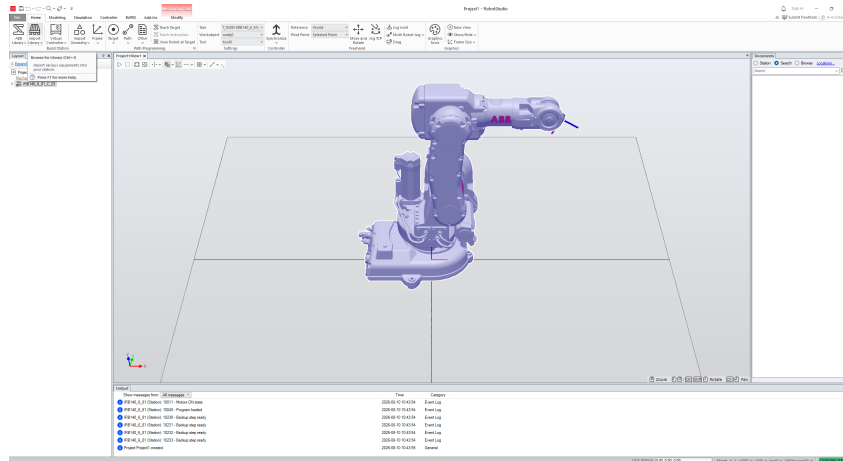


Figure 11: Import library

Next, browse and navigate to the address

This PC\shared\Courses\control\FRTF20_Applied_Robotics\RobotStudio_exercises_2019\Lab1 Pen

After you have found the folder, select the file labeled *rob-table* and import by clicking *Open*. The folder contents are depicted in Figure 12.

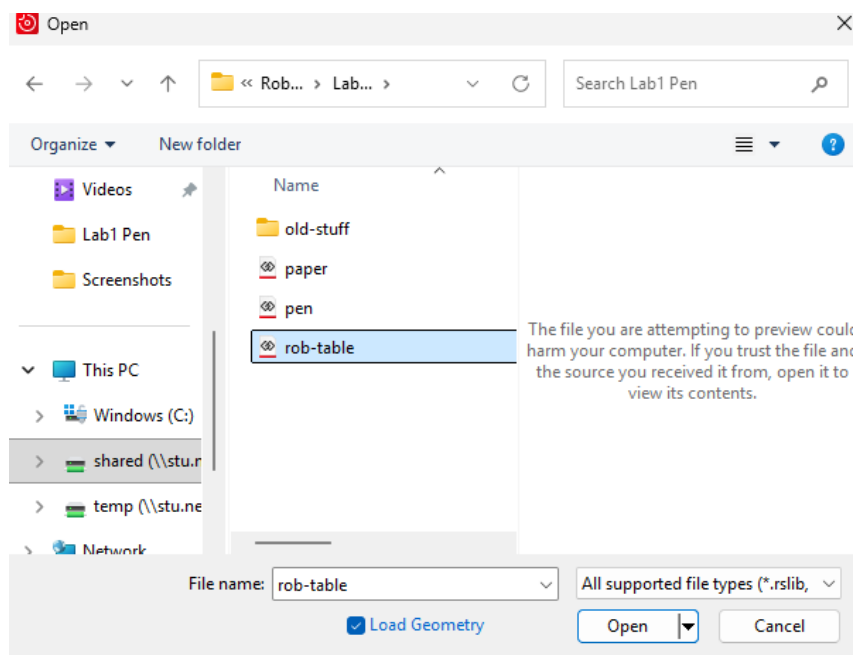


Figure 12: Select the rob-table

After importing the table, the robot and the table will both be at floor level. To place the robot on the table, you need to change the *Task coordinates*. In order to do this you need to go to the *Layout* tab on the left side of the screen. Right click on the robot (IRB140_6.81_C_03 or similar), then go to *Position* and click on *Set Position*, as depicted in Figure 13.

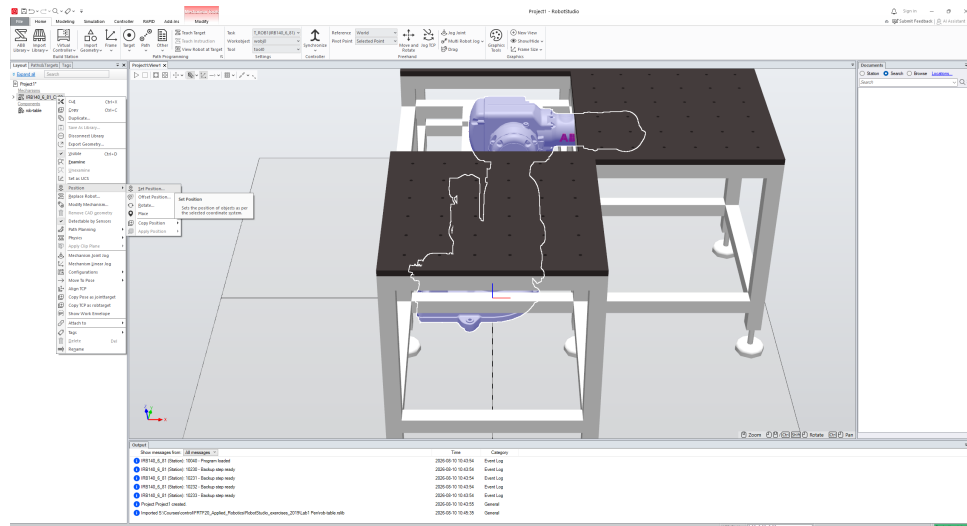


Figure 13: Finding the *Set Position* option (changing the Task coordinates)

A window will pop up, letting you set the frame data. Click on the *Position Z* column and change it to 670 mm (this will put it in the correct place). This is depicted in Figure 14.

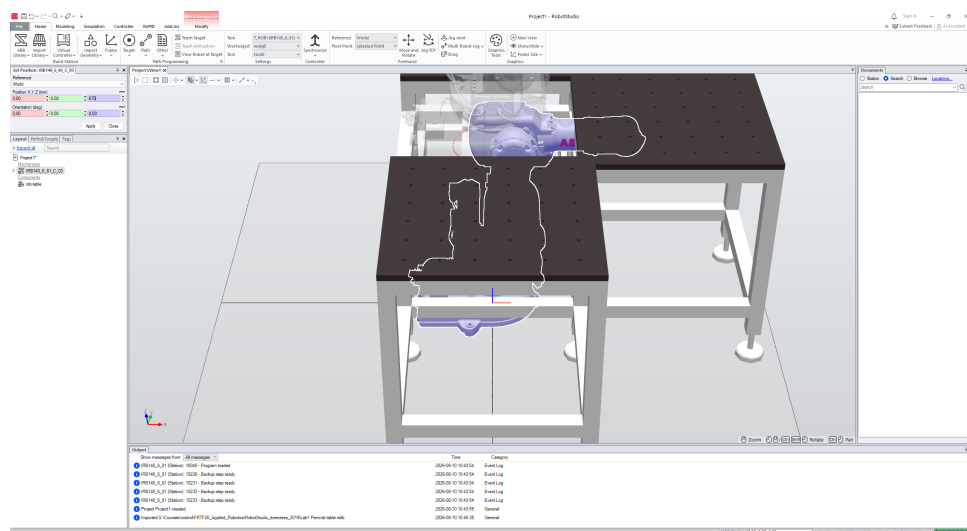


Figure 14: Set the z coordinate of the robot to 670 mm

Leave all other data untouched and click on *Apply*. Confirm that you want to move the *Task frame* when the pop-up (Figure 15) asks you to do so.

Using the same method you used to import the table, previously, you will now have to import the pen and paper models, which are located in the same folder. In the *Layout* view, you should now see the pen and the paper alongside the table.

In the *Layout* tab, drag the pen tool onto the robot (IRB_140....) and confirm that you

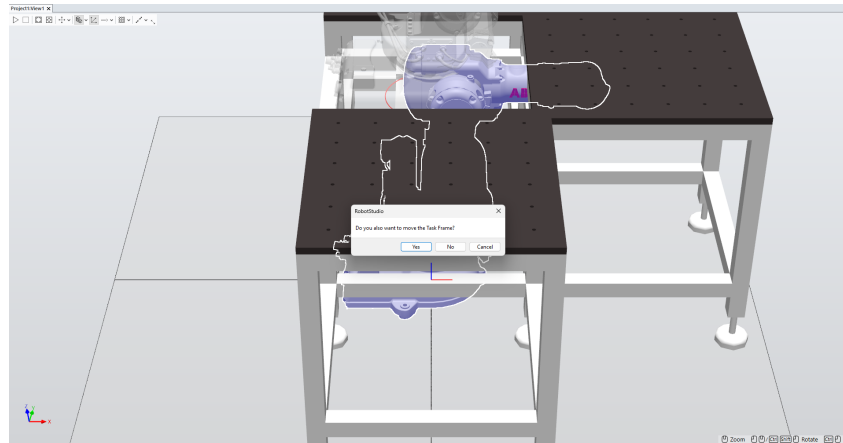


Figure 15: Select "Yes" to move the robot onto the table

want to move and attach the pen to the robot (click *Yes* to the update position question in the pop-up (Figure 16). The pen will now be attached to the 6th link of the robot, and the robot's TCP (Tool Center Point) will be set to the tip of the pen. The paper is already placed in the correct position.

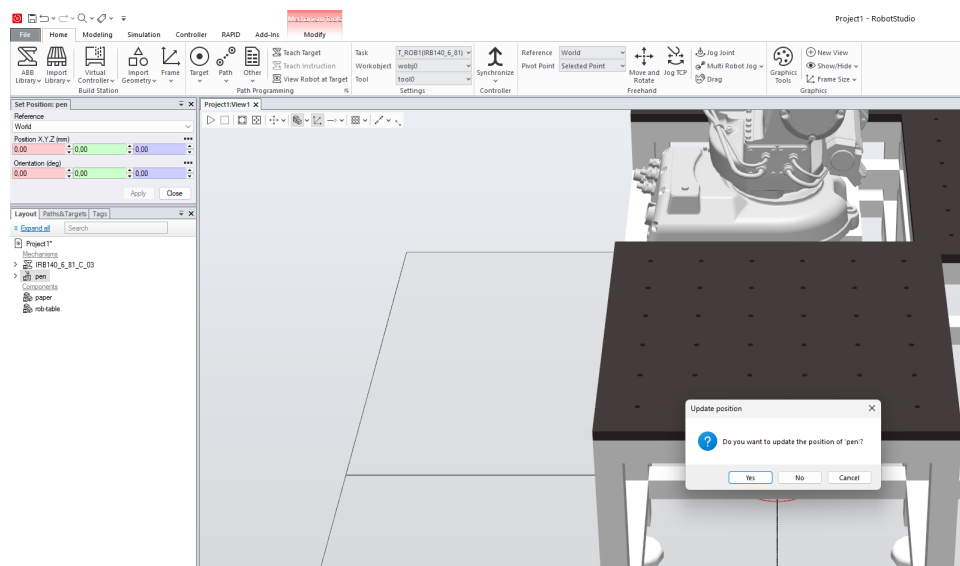


Figure 16: Adding the pen to the end effector

2.3 Defining the home position

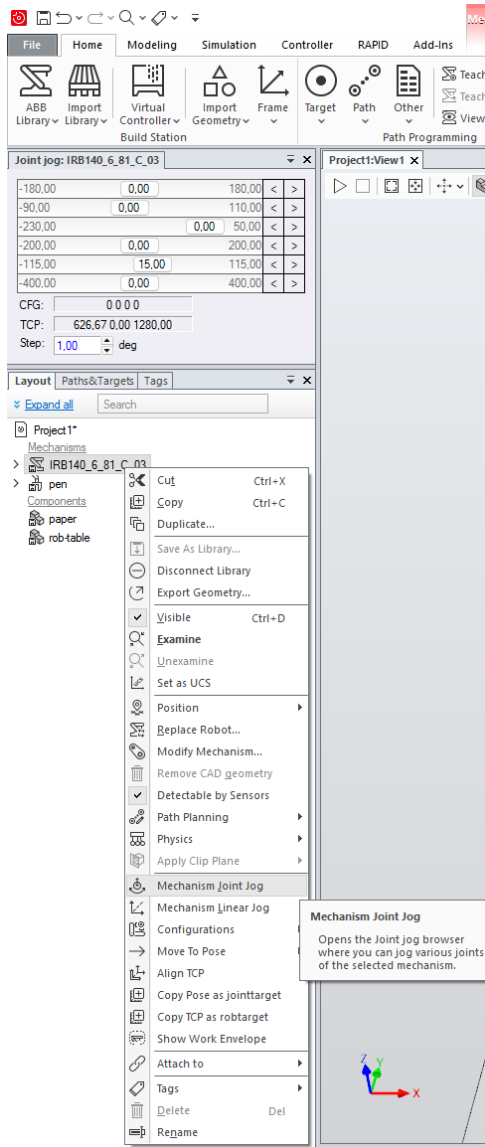
The default *home position* of the robot we have loaded is something called a **manipulator singularity**. You could refer to the lectures for more details, but for now just remember that this is **a position where the inverse kinematics function is ill-defined, and that this is something we must avoid**.

Due to the singularity, you have to manually adjust the joint angles of the robot — the inverse kinematics map does not function in this position.

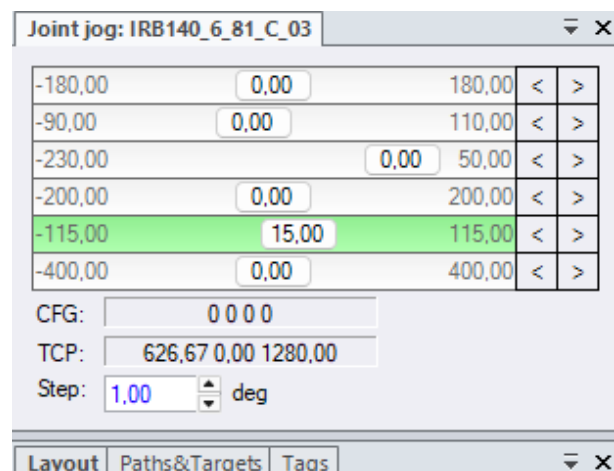
It is important to recognize and avoid **manipulator singularities**, they can and will make programming the task hard if they are not considered!

manually jogging the robot

We will refer to the manual robot's joint changes as **jogging**. To jog the robot, right click on the robot in the *Layout* tab, and select *Mechanism Joint Jog*, as can be seen in Figure 17a. Then adjust the values to the ones you can see in Figure 17b. The resulting pose should be the one from Figure 18.



(a) Selecting *Mechanism Joint Jog*



(b) Joint angle values for a proper home position

Figure 17: Selecting *Mechanism Joint Jog* and the values to be inputed

Mechanism Joint Jog allows you to change the **joint values**, i.e., the angle by which the robot's links are rotated.

There also exists *Mechanism Linear Jog*, with which you can specify the spatial x-y-z coordinates of the end-effector, and the end-effector will move in a straight line to the new coordinates. *Mechanism Linear Jog* uses inverse kinematics to calculate the changes in joint values needed to move it the direction you specified.

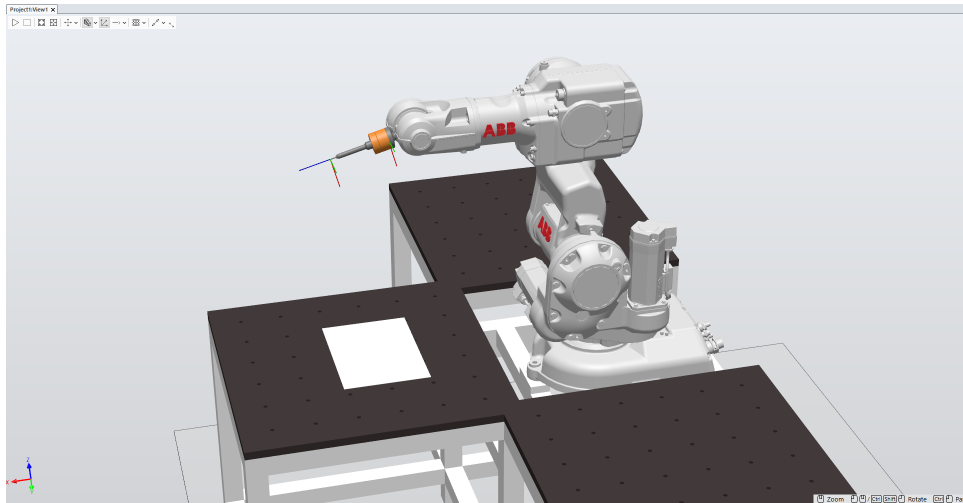


Figure 18: Robot in the proper home position

After you have jogged the robot to the correct position, save it for later use.

We will refer to specific desired robot poses as **targets**. In the *Home* tab, select *Teach Target* to create a target point at the position where the robot is. Refer to Figure 19 to see the location of the button.

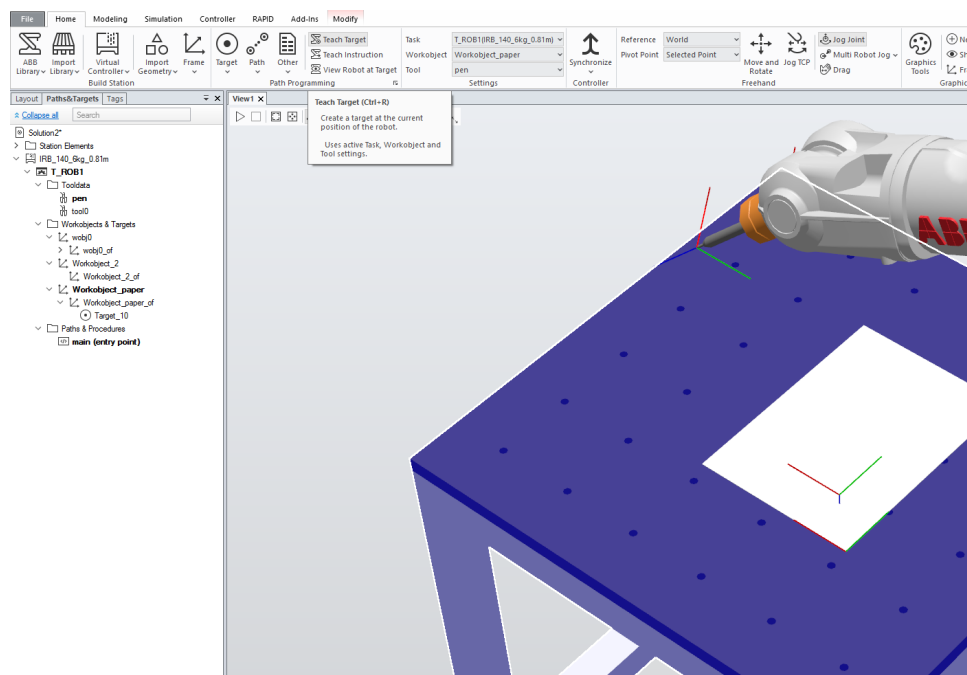


Figure 19: Location of the *Teach Target* button

Once you have taught the target, it will appear in the *Paths & Targets* tab on the menu on the left side of the screen, as indicated on Figure 20. The default name will probably be set to *Target_10*, but we suggest that you rename it to something more informative like *home*, so as to easily differentiate it from other targets you will define later on.

Alternatively, you can teach a target by clicking on the *Target* button. The location of the button is shown in Figure 21. It will allow you to create a target based on its

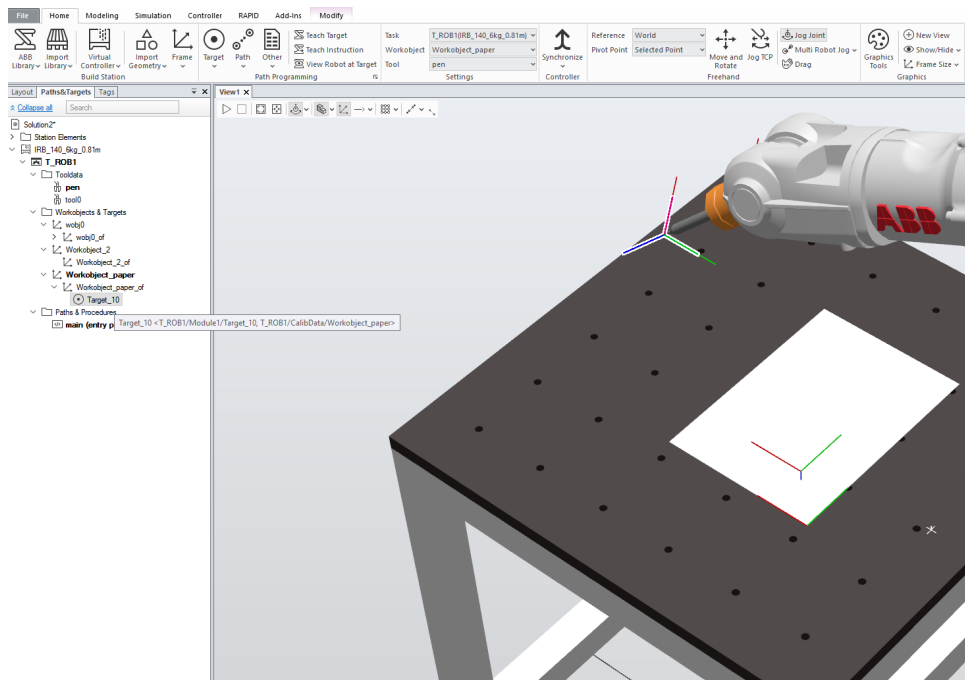


Figure 20: Finding the created target in the *Paths & Targets* tab

x-y-z coordinates. We will utilize this approach shortly.

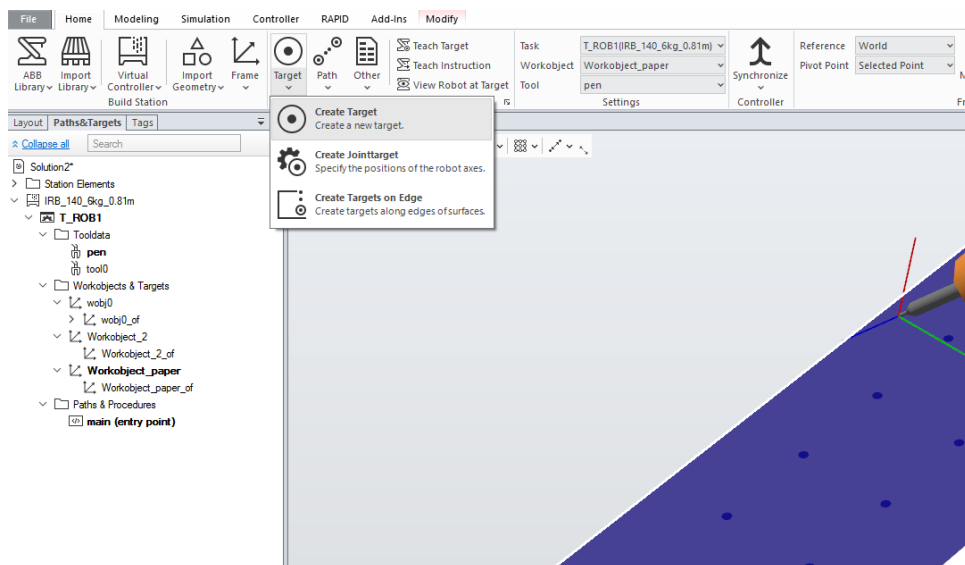


Figure 21: Target saving is very handy, you will use this feature a lot

2.4 Defining a workobject

Your next task is to define a workobject at the location of the paper. Orient the view so you can click on the paper. In order to be able to follow the instructions accurately, make sure to orient the view **exactly as in Figure 22**. Once you are sure that you are looking at the correct corner of the paper, feel free to zoom in to it as we will be selecting it in the following step.

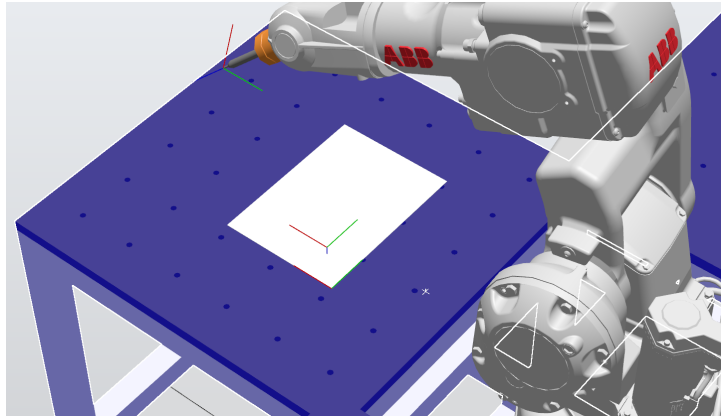


Figure 22: We will be creating a workobject in the lower left corner of the paper

To make things less complicated, select *Snap End* on the toolbar on top of the simulation window to be able to easily click onto edges. The toolbar can be quite hard to see as it turns transparent, but hovering the mouse over it should reveal it. Refer to Figure 23 to find the button.

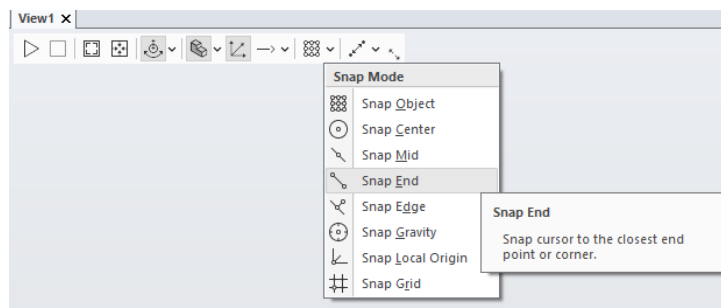


Figure 23: Various options for cursor snapping

Select the indicated corner, click *Home* → *Other* and then *Create Workobject* as shown in Figure 24.

A window will pop up on the left side of the screen asking you to fill in some data for the new workobject. Click on the small downward arrow on the box titled *Values...* located at *User Frame* → *Position x,y,z*. Click on the red square, and then click on the corner of the paper in the solution view. If done correctly, the positions should now be (450,125,120.5) as indicated on Figure 25. Click on *Accept*. The result is indicated in the Figure 25 (you might need to expand *Position x,y,z* to get the same view).

By default, the Z-axis is pointing up. The pen tool's Z-axis is pointing in the direction of the pen (which will be down while drawing). We have to make sure our coordinate frame is properly oriented. The axis of the *User Frame* should point down.

between drawings when programming the motion to achieve the drawings.

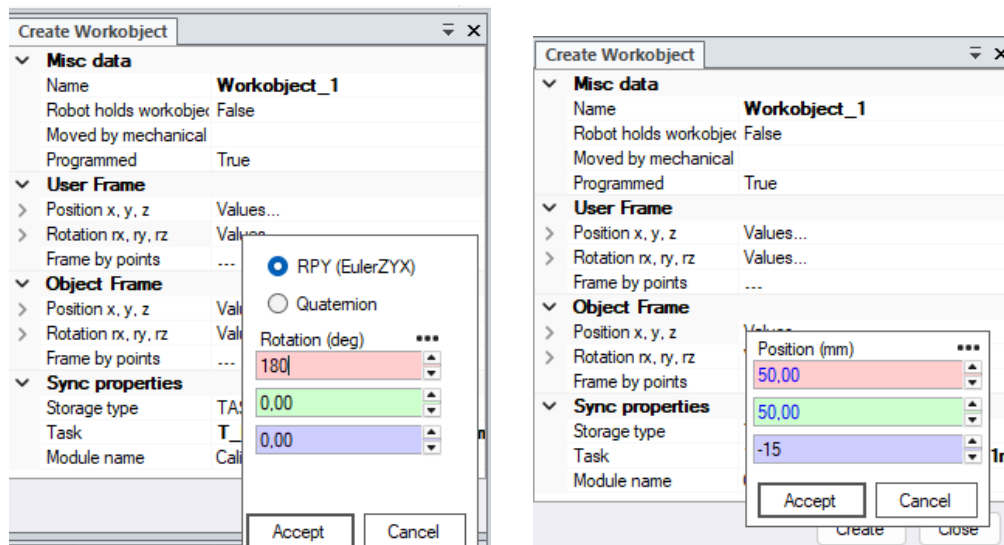


Figure 26: Further modifications of the *Workobject's* User frame and Object frame

Create the workobject and close the window.

If you realize you have made a mistake later on, you can always go back and fix it using the *Modify Workobject* option which you obtain by right clicking on it in the *Paths & Targets* menu of the left side of the screen. Make sure you got all of the axis orientations right by comparing them to Figure 27.

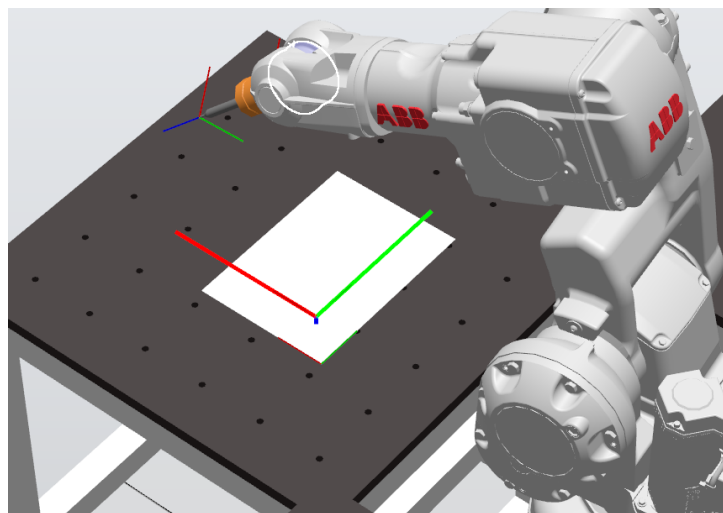


Figure 27: Double check the orientation of the arrows

Now is a good time to **save your current implementation**. Remember to use a local folder!

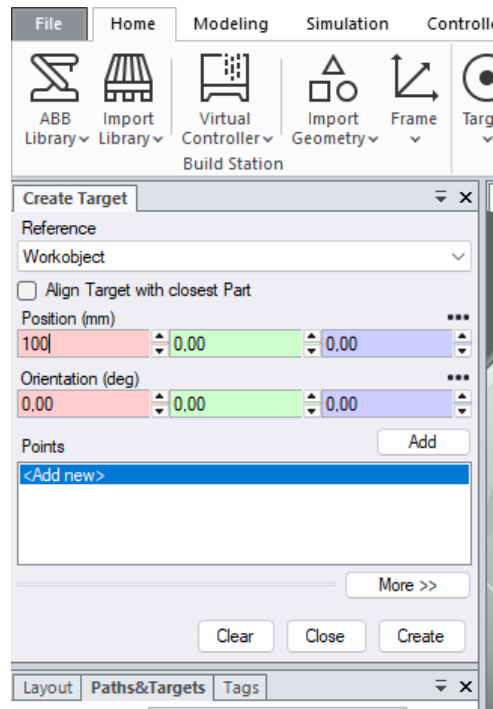


Figure 29: Define target coordinates in the *Workobject Reference*

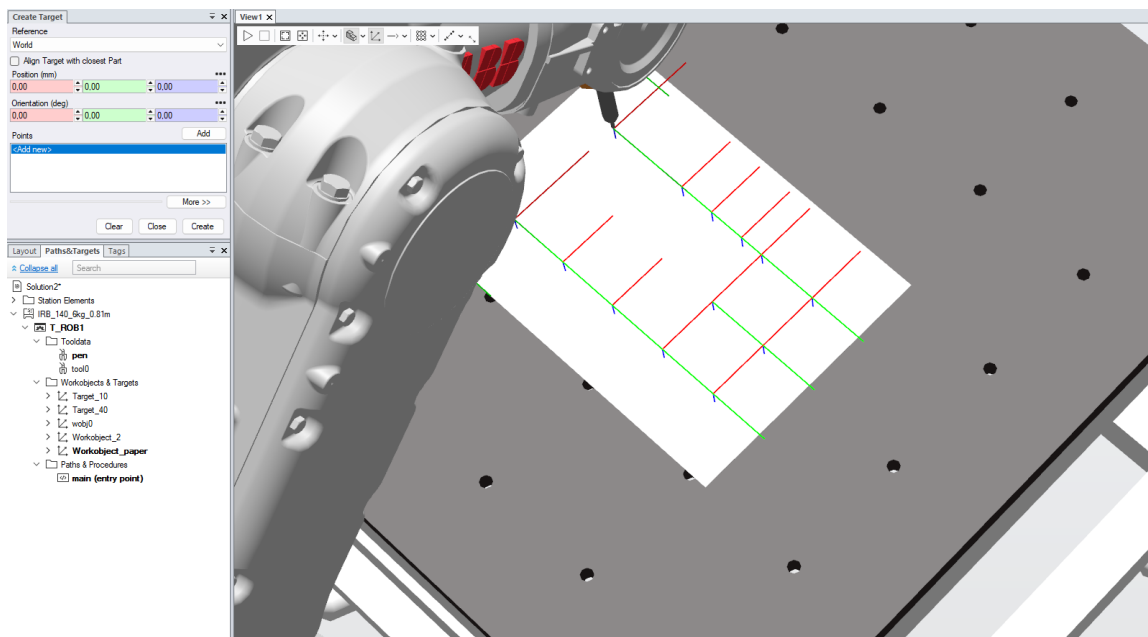


Figure 30: Visible targets on the workobject (paper)

path into something suitable (in this text we will use *Path_L*). Next, drag your targets to the created path, and finish it with the home position. To inspect the path right-click on it and select *Move Along Path*. You can (re)calculate the movements between points by right-clicking on the path and selecting *Auto Configuration* and selecting one of the options.

Always run *Auto Configuration* after defining a path to ensure that the robot can actually execute the path.

Sometimes, especially when multiple paths are defined, configuration errors might occur. If this happens, you need to modify some targets' configuration by right-clicking on the problematic target, then *Configurations* and select one similar to (or the same as) the previous target's, as shown in Figure 31. Repeat this until necessary.

2.6 Creating a RAPID program

In order for the (real) robot to execute the defined path, you need to translate the path into RAPID code.

Functionally, this is similar to compiling code into executables that can be run on a computer (we are now compiling from the RobotStudio GUI into RAPID code, which will later be compiled into executables on the robot).

You will also edit some of the generated RAPID code. If you wish to learn more about RAPID, you can consult the *RAPID Reference Manual*, which is available online.

To create a RAPID program of your work, click on *Synchronize* → *Synchronize to RAPID* as indicated on Figure 32.

You will get a pop-up window as in Figure 33.

Remember to synchronize to RAPID after you make changes in the GUI, otherwise you would not update the RAPID program that goes to the robot.

You can also make changes directly in the RAPID code through the *RAPID-editor*. In this case click on *Apply* after making the changes and then click *Synchronize to Station* to actually apply the changes.

2.7 Analyzing and editing RAPID code

We will now look at the generated RAPID code. Start by clicking on the *RAPID tab* (located in the topmost menu (same as *Home*, *Modeling*, *Simulation*, *Controller*,...,*Add-Ins*) and expanding the tree of the robot task so that you can see *Program Modules* as shown on Figure 34a. Now you need to select *Module1* and then right click, and select *RAPID Editor* icon, as shown in Figure 34b.

You should now see the generated RAPID code in the RAPID editor as shown in Figure 35. **Even if you are able to see the code, you still need to right-click on a module and select *RAPID Editor* to make and save changes to it.**

Before editing the code, right-click on a module and select *RAPID Editor*. For your changes to RAPID code to take effect, you need to apply them by going to *Apply* → *Apply Changes* (as shown in Figure 36) and then synchronize by going to *Synchronize* → *Synchronize to Station* (also available in the *RAPID tab*).

A *module* is just a file containing RAPID code. A *procedure* is what is in other programming languages also referred to as a function or a (sub)routine. In RAPID it starts with *PROC procedure_name()* and ends with *ENDPROC*.

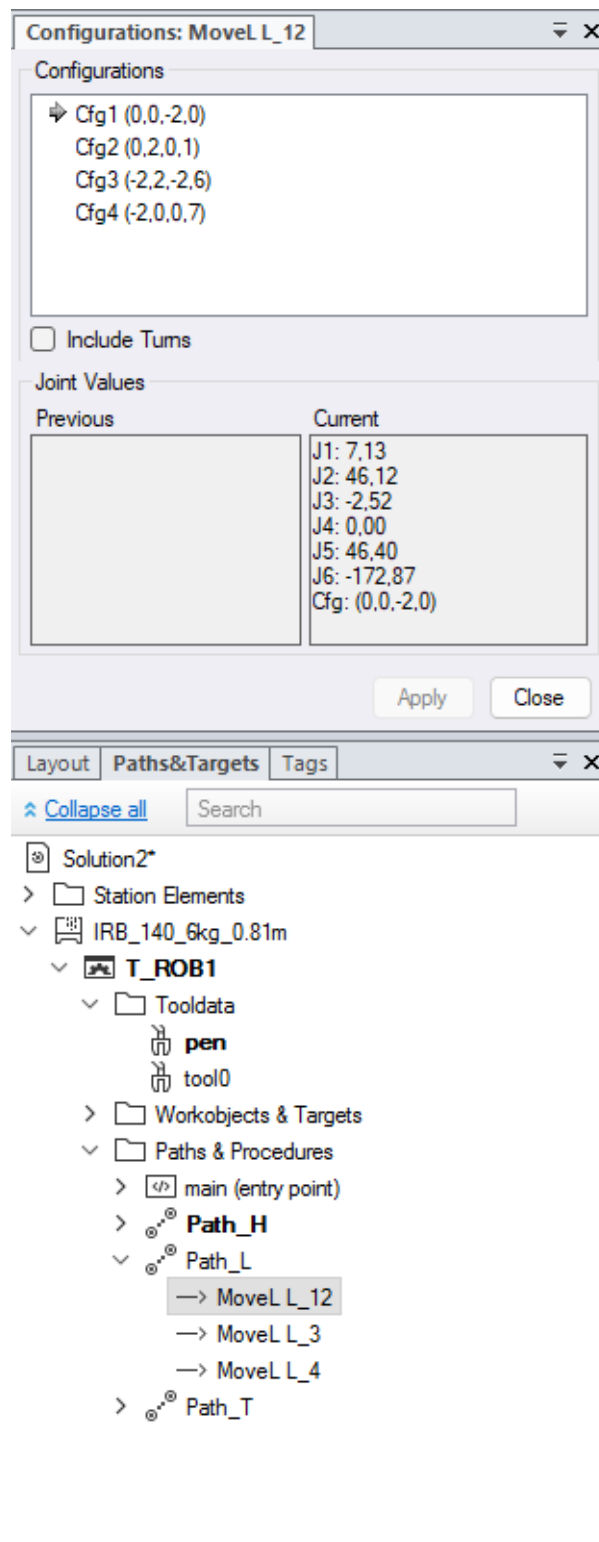


Figure 31: Inspecting the created path. The column *Current* shows joint positions (rotations) for each configuration. Ideally, joint values of nearby targets should be similar. You can see how the robot moves by clicking on the targets one by one.

You will now edit the *main* procedure. Depending on the RobotStudio version, you may find it in *Module1* or in the *MainModule*. It does not matter where it is as you can call procedures from other modules (if they are loaded). You can put it in a new

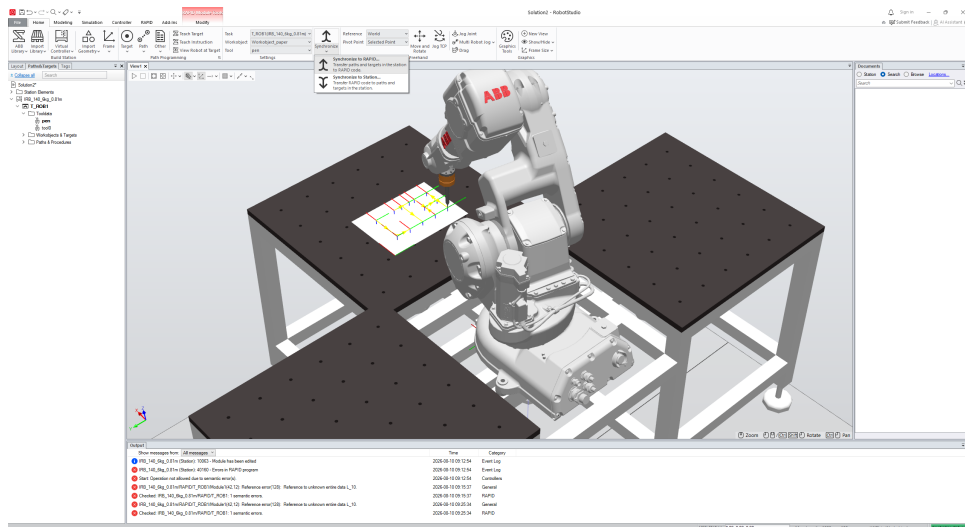


Figure 32: Selecting *Synchronize to RAPID*

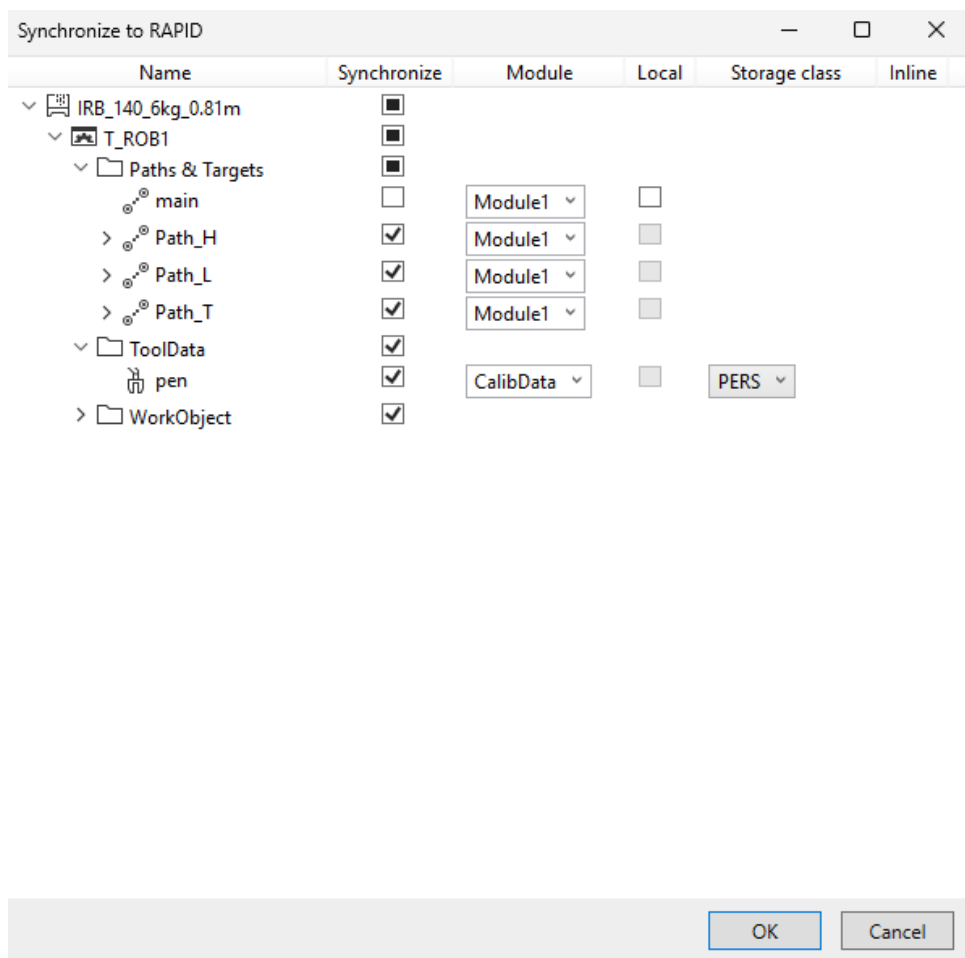


Figure 33: Pop-up window for selecting what to synchronize to RAPID. If you are changing only a specific part of the program, you can select only that one for the synchronization.

module if you want to, or edit it where you found it. Refer to Figure 37 to find the button for a new module.

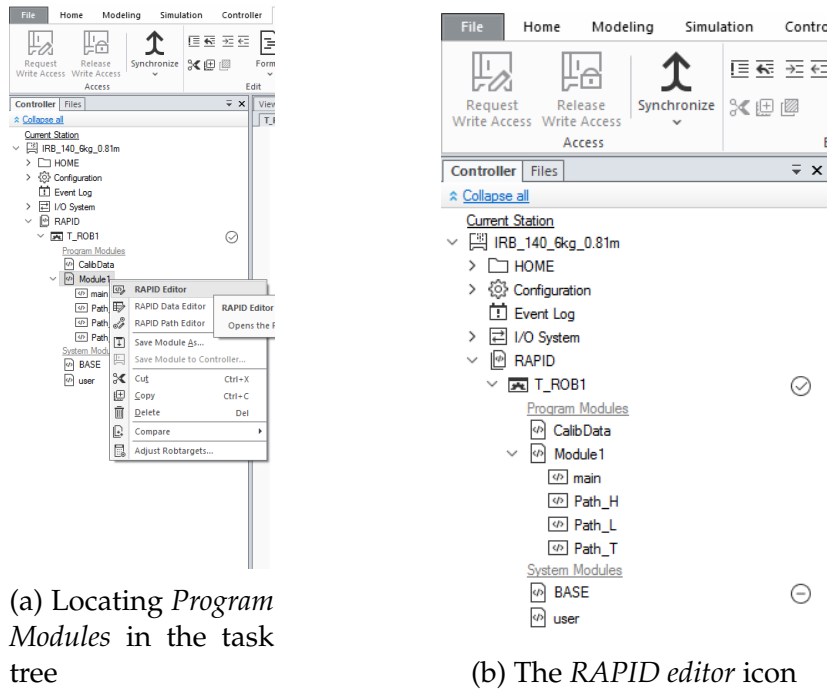


Figure 34: Finding *Module1* and going to the *RAPID editor*

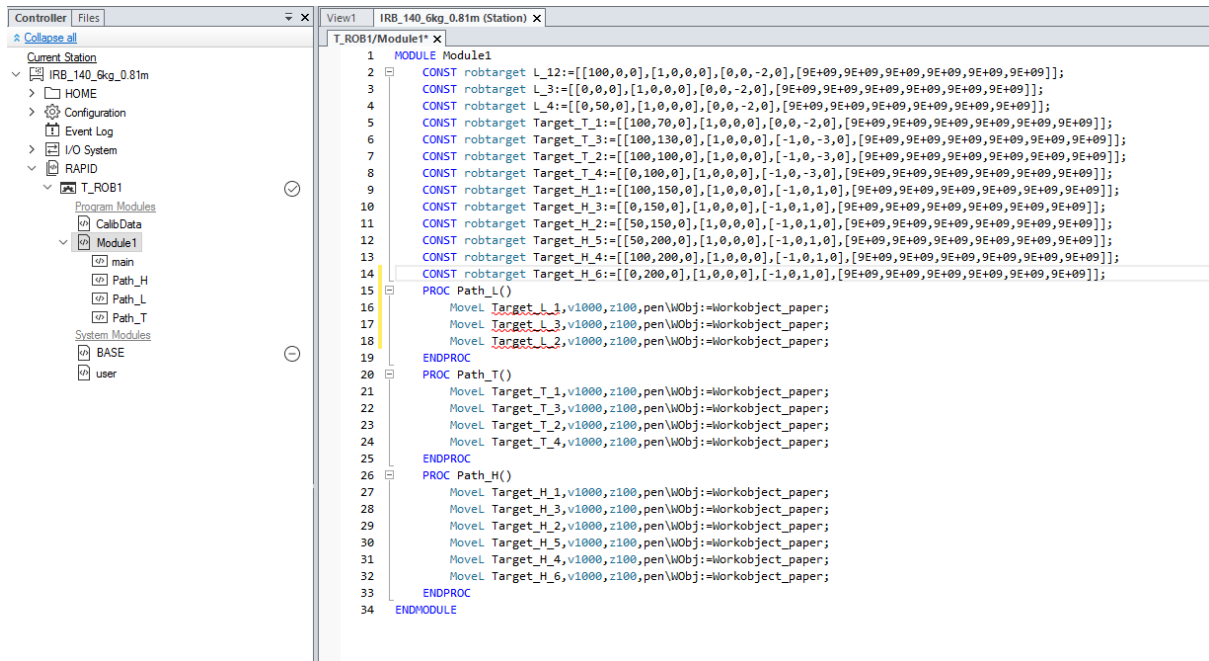


Figure 35: Code in RAPID editor you get when from right clicking on *Module1*

Note: If you have not found the *main* procedure anywhere, define it yourself.

Once you have found the *main* module, in it simply add the procedure calls to paths you have defined. Do this by putting in a line like *Path_Name*; for every path. Put the commands between *PROC main()* and its *ENDPROC*.

Once you have put in your paths, save them via *Synchronize to Station*. Since you have only made changes to the *main* procedure you only need to select it for synchronization, but **selecting everything also works**. The *Synchronize to Station* pop-up can

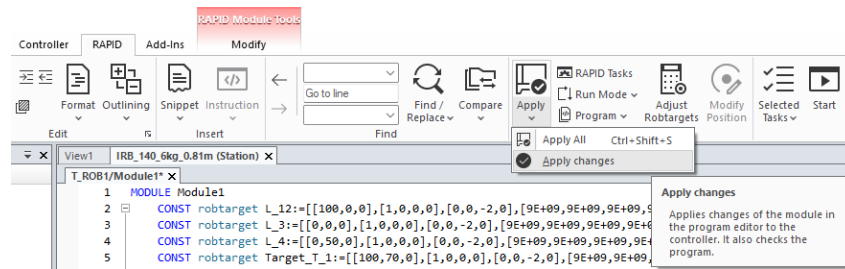


Figure 36: Main module code and applying changes

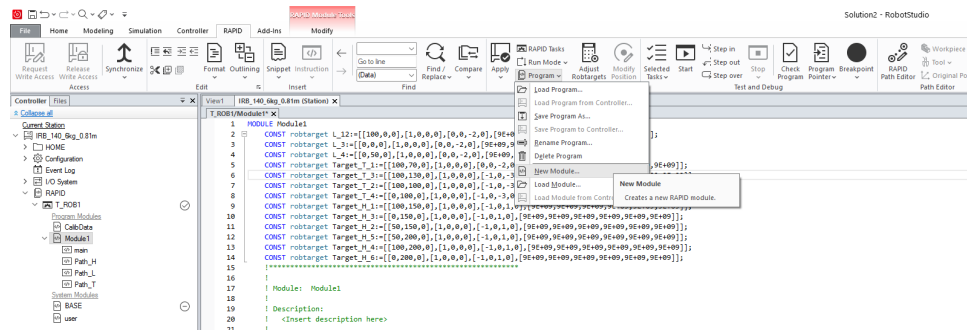


Figure 37: Creating a new module

look as shown in Figure 38.

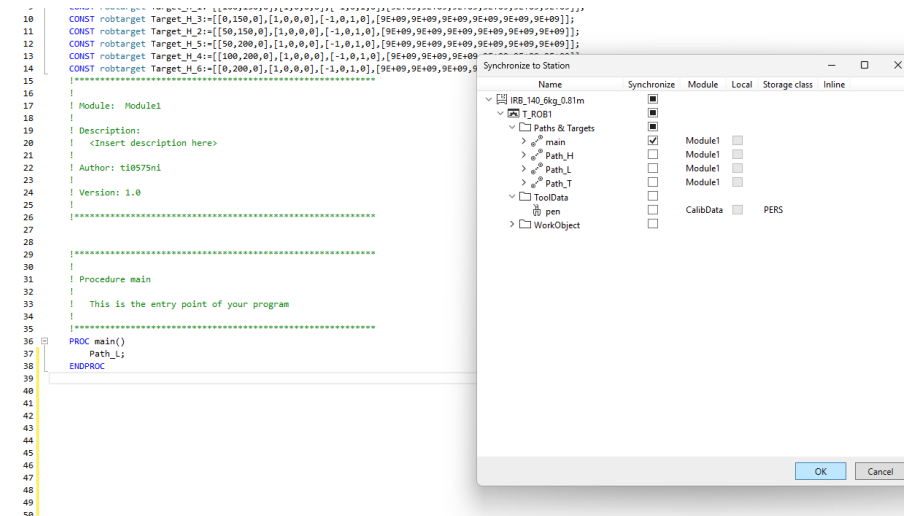


Figure 38: Synchronizing changes made to *main*

Click OK in the *Synchronize to Station* pop-up and go to the module with you paths, which is called *Module1* and contains *Path.L* in this text.

Note that the *main* procedure can be in the same, or in a different module than *Path.L*, as long as these modules are loaded to the controller, it will work just fine.

In RAPID code, you can call procedures from other modules **if they are loaded**.

We can now do some preliminary editing of the code for the drawings. To make a drawing, we must move down to **Z value 0** — remember that the *User Frame* was

put above the paper! This can be done using an offset instruction which is called *Offs* in RAPID code. In the editor you can copy and paste as usual. In this text, we rearranged the drawing so that it starts at *Target L_10*, goes to *Target L_20*, and finishes at *Target L_30*. Look at Figure 40 to see what name corresponds to which coordinate.

Note: If your *Target L_20* corresponds to the guide's *Target L_10*, you can either rename your target, or change the lines accordingly. Be sure to synchronize to RAPID if you decide to change the names of the targets.

Change the *v* (velocity) and *z* (zone) arguments so that the velocity during drawing is 100 and *z* is 1, except for the home position, which can have *z*100.

This is the appropriate speed for the robot — going too fast poses a safety risk.

What the zone means is explained graphically through Figure 39, but intuitively it refers to the allowable error to the target position.

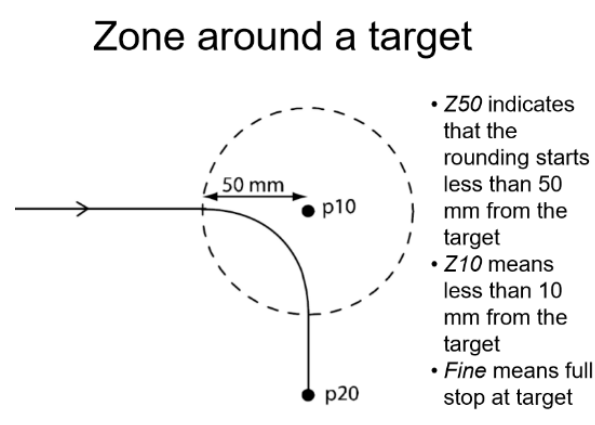


Figure 39: What the *z*-value, or zone means

Figure 40 is an example of the final code for the first letter. **Finish the code for the other two letters.**

```

PROC Path_L()
  MoveJ L_10,v100,z1,pen\WObj:=Workobject_paper;
  MoveL Offs(L_10,0,0,15),v100,z1,pen\WObj:=Workobject_paper;
  MoveL Offs(L_20,0,0,15),v100,z1,pen\WObj:=Workobject_paper;
  MoveL Offs(L_30,0,0,15),v100,z1,pen\WObj:=Workobject_paper;
  MoveL L_30,v100,z100,pen\WObj:=Workobject_paper;
ENDPROC

```

Figure 40: A finalized version of the code for **the first letter**. You should have the target names you have defined. Make sure they match the ones *PROC Path_L()*. **Finish the code for the other two letters.**

Remember to *Apply Changes* and then *Synchronize to Station*! The pop-up should look similar to Figure 41 (make sure you select all of the boxes as shown in the figure).

Click on *Apply changes* as indicated in Figure 38 and make sure all modules where you made changes are selected. If any errors pop-up they will be indicated and you need to correct them in order to proceed. When all indicated error messages have been corrected, click on *Synchronize* → *Synchronize to Station* (in the RAPID-tab) and

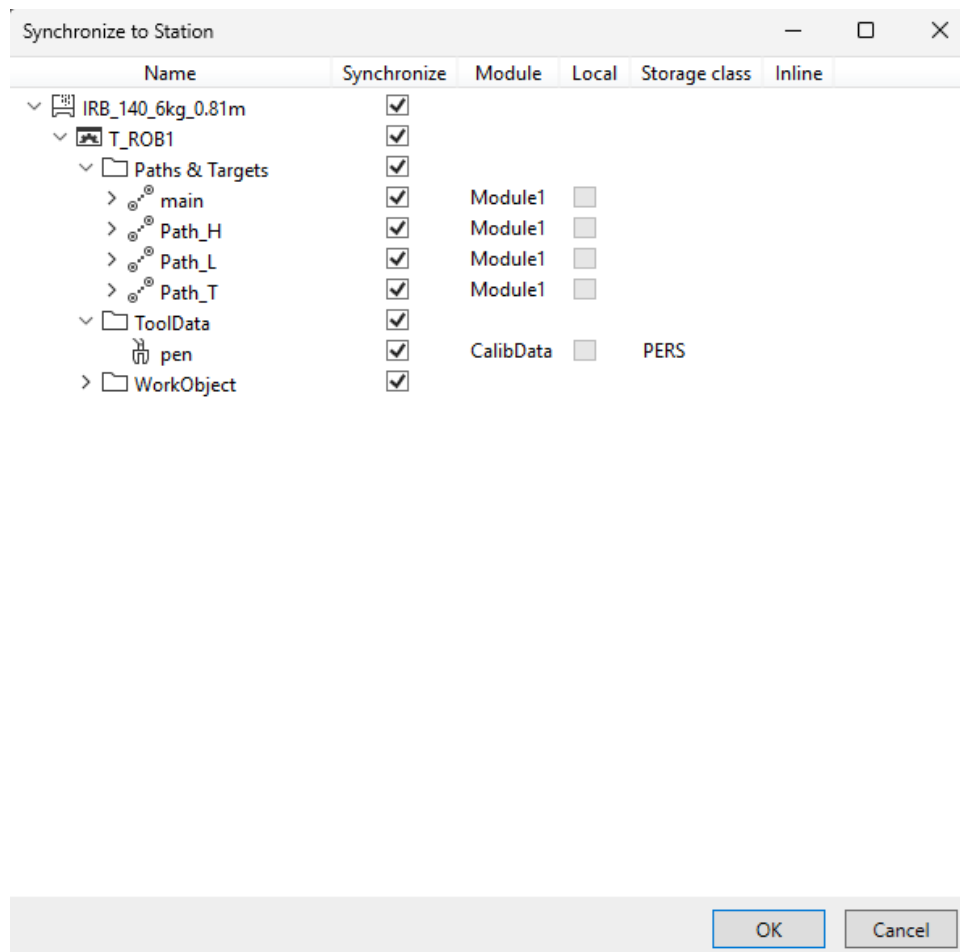


Figure 41: *Synchronize to Station* pop-up with all the defined paths

click OK in the pop-up window.

Click on the *Simulation* tab and run the simulation by clicking the *Play button*. You might get an error during the simulation. This is related to a common problem when making long linear movements.

After finishing the code for all letters, change the move instruction to *MoveJ* to go to the first target for all letters. Also make the move from the last target in the last letter towards the home position into a *MoveJ*. This needs to be done because sometimes the program fails if *MoveL* is the first or the last instruction. When the robot moves between letters using *MoveJ*, make sure that it does not collide with the table when moving.

Note: *MoveL* and *MoveJ* are programmed equivalents of *Mechanism Linear Jog* and *Mechanism Joint jog* we discussed previously. For additional information on the difference between *MoveL* and *MoveJ*, see p. 241 and p. 249 in *RAPID Reference Manual System Data Types and Routines On-line*, which is available online.

Finally, go to the *RAPID* tab and *Synchronize* → *Synchronize to RAPID*. Notice how the *RAPID* code changes. Run the simulation again (you possibly need to make a reset before, which is done by pressing *Reset*). Change your view and make a close-up to see the movement on the paper.

During editing save your work **frequently**! If RobotStudio unexpectedly stops working try *Restart (P-start)* and synchronize only *main* and one of the paths.

The exact same thing goes if synchronization stops working. However, in this case there is most likely a configuration error, which is most likely due to a mistake in the code.

Do not forget to use *Auto Configuration* and check reachability. If the problem persists, try adding additional instructions to change the robots pose.

2.8 Simulation options and an additional assignment

Add more targets to your *Workobject* and create some additional drawings or letters. Create a path for each new drawing and edit the RAPID code to add new procedures in *main*.

You can define new paths graphically or directly in RAPID code. To better view your drawing, you can activate a trace of where the TCP has moved: *Simulation* → *TCP Trace*, as indicated in Figure 42.

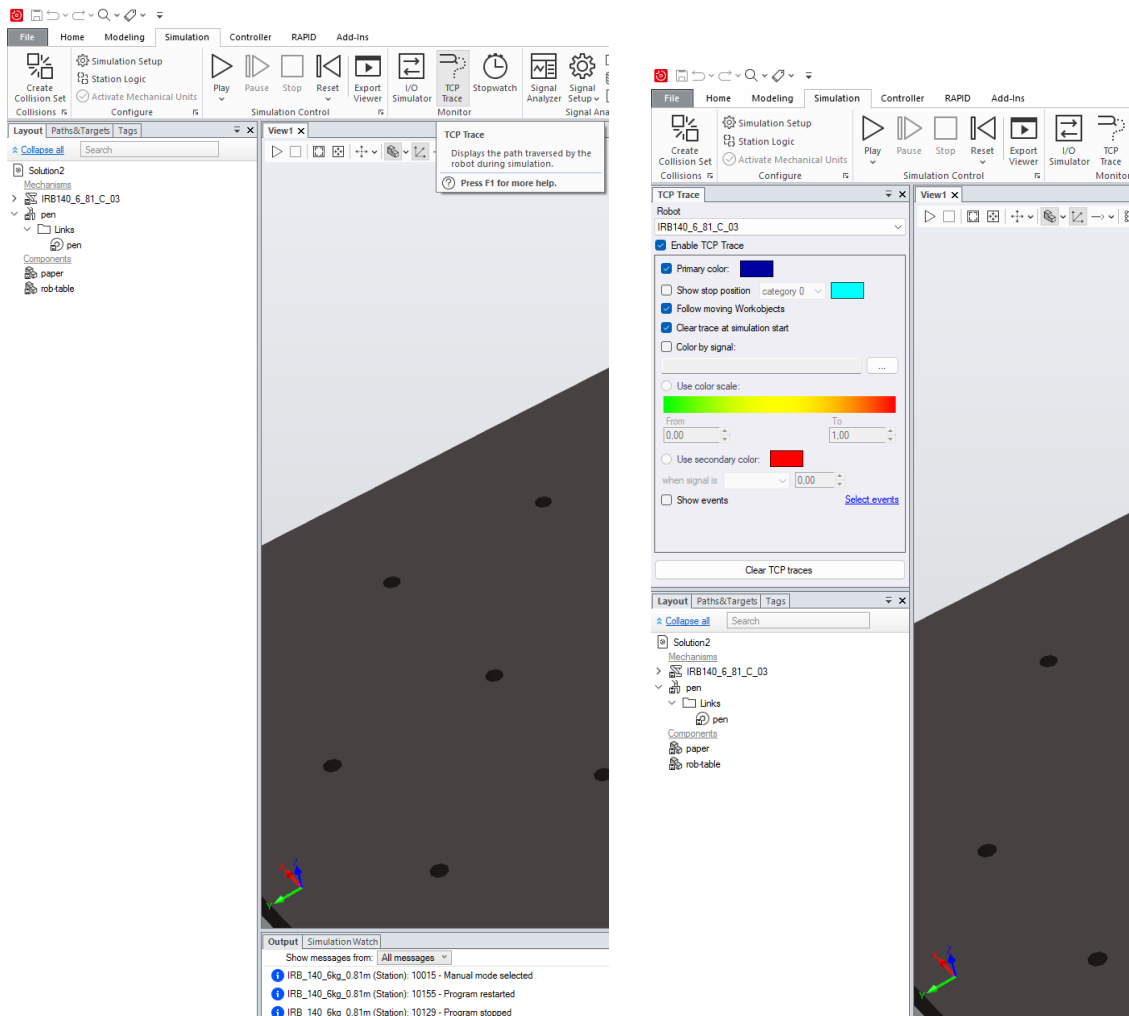


Figure 42: Turning on TCP trace and setting the trace to a color of your choosing

3 Testing your program on the real robot

Once you are satisfied with the simulation, **show** it to the teaching assistant. You **must** always do this before running your program on the real robot!

To run your program on the real robot you need to save it, and then put it on a USB stick which you will get from the teaching assistant. The teaching assistant will load and run your program on the robot.

Make sure you save **all needed modules**, not just the *main* program. To save the executable program to the USB, go to the *RAPID* tab and right-click on *T_ROB1* → *Save Program As...* as indicated on Figure 43.

To save your work so that you can transfer it between computers (for example in between different sessions), save it as a *Pack&Go* file (.rspag). You can pack and unpack via the *Share* menu as indicated on Figure 44.

When saving as a *Pack&Go* file, remember to **copy it from the TEMPORARY local disk to your home folder before leaving the exercise.**

When packing and unpacking the *Pack&Go* file you might get error messages if you

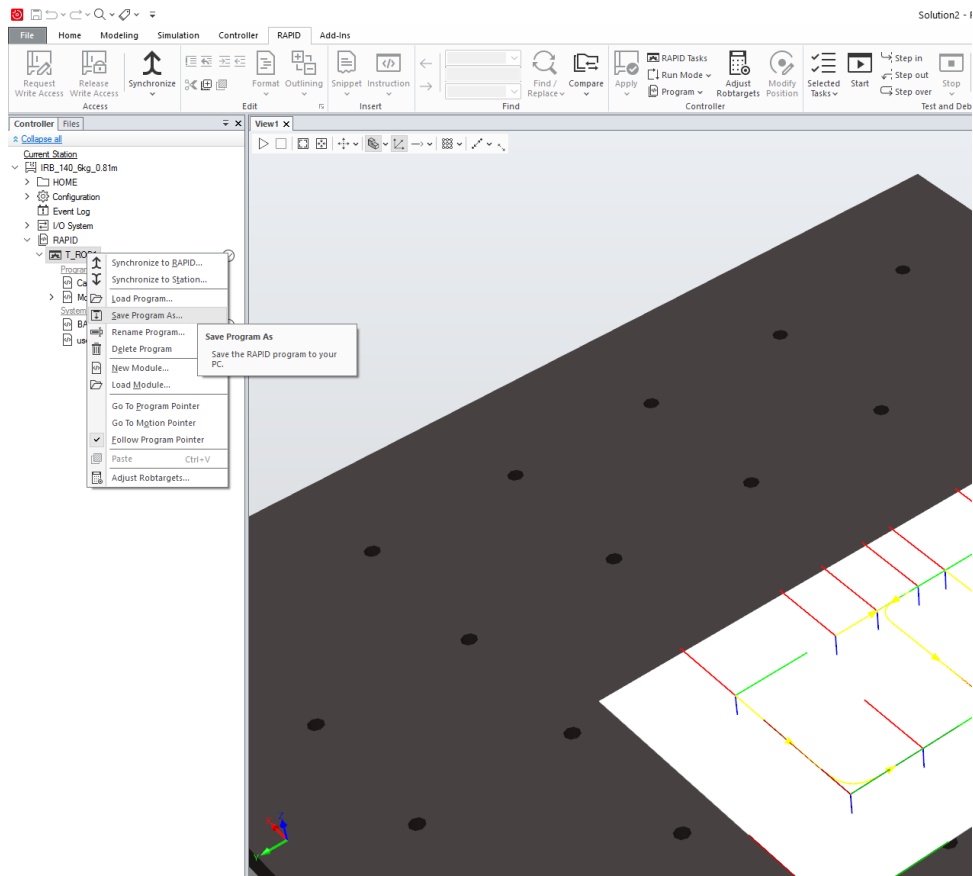


Figure 43: Finding *Save Program As...* to save the program

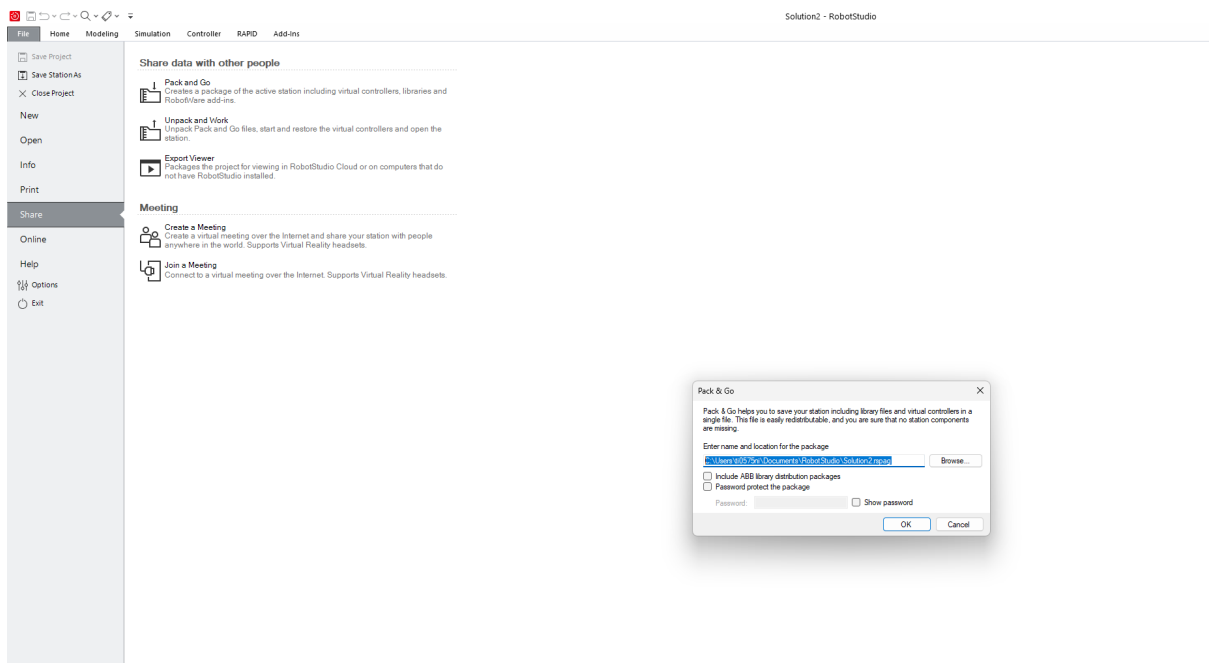


Figure 44: Saving everything as a *Pack&Go* file

select the wrong network drive.

This will probably look something like in Figure 45 or Figure 46. Copy the *Pack&Go*

file to your temporary local disk and select that version when you try to open the file. You might also need to create a new subfolder in order to open the unpacked file.

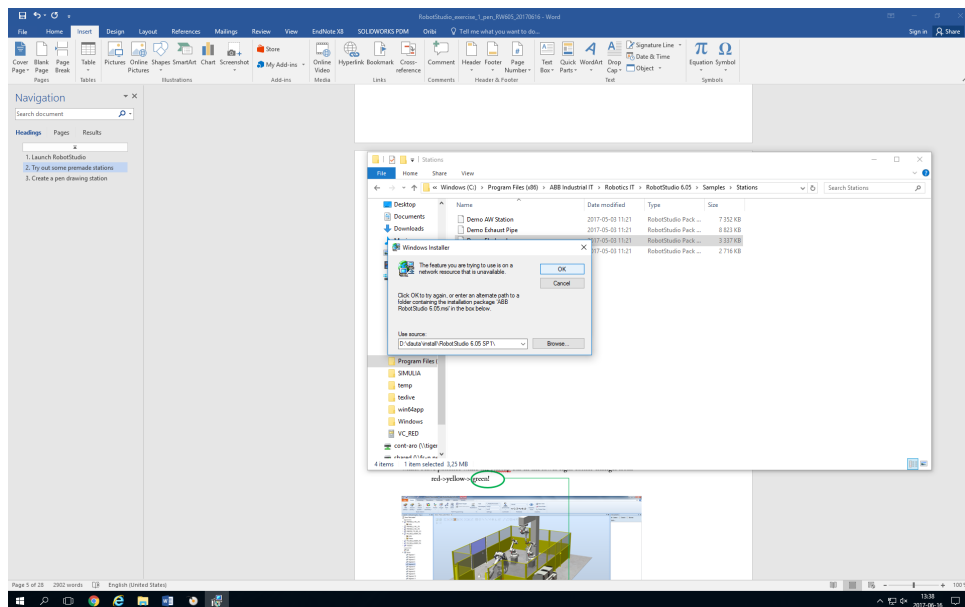


Figure 45: A possible error when unpacking

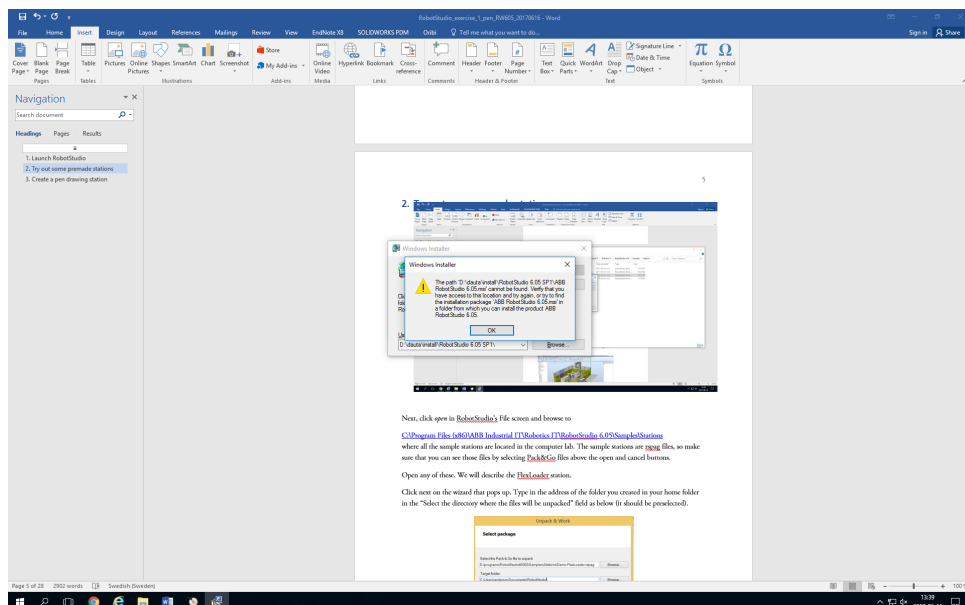


Figure 46: Another possible error when unpacking



LUNDS UNIVERSITET
Lunds Tekniska Högskola